P3 – Pseudocodes

Compiled by Zuhaib Abdul Khalique

Recursion

Recursion is a programming technique where a function **calls itself** to solve a smaller version of the same problem

A recursive solution has two main parts:

1. **Base Case –** stops the recursion
2. **Recursive Case –** reduces the problem and calls the function again

Basic Structure of a Recursive Function:

```
FUNCTION Func_Name(Parameter_Name : Parameter_DataType)
        IF Base_Case_Condition THEN
                RETURN Base_Case_Value // Base Case
        ELSE
                RETURN Func_Name(Smaller_Input) // General/Recursive Case
ENDFUNCTION
```

Why Use Recursion?

Recursion is good when a problem can be broken down into smaller subproblems, such as:

- Tree/Graph Interval
- Divide-and-conquer algorithms
- Mathematical Sequence (Factorial, Fibonacci)
- Backtracking problems

The **base case** prevents infinite recursion, specifies the simplest input & directly returns an answer

The **general(recursive) case** breaks the problem into smaller parts and moves towards the base case

Recursion may reduce code & it can perform loop operations without using them.

Avoid Recursion when:

- Iteration is simpler
- Too deep recursion may cause stack overflow
- Performance is critical (e.g: naïve Fibonacci)

Ex # 01 - Factorial using Recursion:

Without Recursion:

```
FUNCTION Factorial(Num : INTEGER) RETURNS INTEGER
    IF Num < 0 THEN
        RETURN -1
    ELSEIF Num = 0 THEN
        RETURN 1
    ELSE
        Result ← 1
        FOR Index ← Num TO 1 STEP -1
            Result ← Result * Index
        NEXT Index
        RETURN Result
    ENDIF
ENDFUNCTION
```

With Recursion:

```
FUNCTION Factorial(Num : INTEGER) RETURNS INTEGER
    IF Num < 0 THEN
        Answer ← -1 // Base Case
    ELSEIF Num = 0 THEN
        Answer ← 1 // Base Case
    ELSE
        Answer ← Num * Factorial(Num-1) // General/Recursive Case
    ENDIF
    RETURN Answer
ENDFUNCTION
```

Call Number	Function Call	Num	Answer	RETURN
1	Factorial(4)	4	4 * Factorial(3)	
2	Factorial(3)	3	3 * Factorial(2)	
3	Factorial(2)	2	2 * Factorial(1)	
4	Factorial(1)	1	1 * Factorial(0)	
5	Factorial(0)	0	1	1
(4)	Factorial(1)	1	1 * 1	1
(3)	Factorial(2)	2	2 * 1	2
(2)	Factorial(3)	3	3 * 2	6
(1)	Factorial(4)	4	4 * 6	24

In the table above, from row 1 to 4 its winding, row 5 is base case & from row 6 to 9/(4) to (1) its unwinding

Ex # 02 – Binary Search using Recursion

Without Recursion:

```
FUNCTION BinarySearch(Num : INTEGER) RETURNS INTEGER
    DECLARE Top, Bottom, Middle : INTEGER

    Top ← 1
    Bottom ← 10

    WHILE Top <= Bottom DO
        Middle ← INT((Top + Bottom)/2)
        IF Num = MyList[Middle] THEN
            RETURN Middle
        ELSEIF Num > MyList[Middle] THEN
            Top ← Middle + 1

        ELSE
            Bottom ← Middle – 1
        ENDIF
    ENDWHILE

    RETURN -1
ENDFUNCTION
```

With Recursion:

```
FUNCTION BinarySearch(Num, Top, Bottom) RETURNS INTEGER
    IF Top > Bottom THEN
        RETURN -1 // Base Case 1
    ENDIF
    Mid ← (Top + Bottom)/2 // Base Case 2
    IF MyList[Mid] = Num THEN
        RETURN Mid
    ELSEIF Num > MyList[Mid] THEN
        RETURN BinarySearch(Num, Mid+1, Bottom) // General Case 1
    ELSE
        RETURN BinarySearch(Num, Top, Mid-1) // General Case 2
    ENDIF
ENDFUNCTION
```

User-Defined Data Types

- A data type based on an existing data type or other data types that have been defined by a programmer.

There are two types of user defined data types:

1. **Composite:** A data type that refers to any other data type in its type definition is a composite data type.

   The referenced data type may be non-composite, composite or primitive(fundamental data types for e.g: STRING, INTEGER etc.

2. **Non-Composite:** A data type that does not reference any other data types.

There are three types of composite data types:

1. **Record:** Records are composite data types formed by the inclusion of several related items that may be of different data types. This allows a programmer to refer to these items using the same identifier, enabling a structured approach to using related items. A record will contain a fixed number of items.
2. **Sets:** A set is a given list of unordered elements that can use set theory operations such as intersection and union. A set data type includes the type of data in the set.
3. **Class:** A collection of **Methods**(Functions & Procedures) and **Attributes/Properties**(Variables)

An object is defined from a given class, several objects can be defined from the same class.

There are two types of non-composite data types:

1. **Enumerated:** A non-composite data type defined by a given list of all possible values that has an implied order.

As it's name suggests it has come from numbers and the elements are stored as numbers actually. This is rarely used. Examples include drop down list

2. **Pointers:** A non-composite data type that uses the memory address of where the data is stored.

Composite Data Types

Record

```
// Example of defining a record datatype
TYPE Students
        DECLARE StudentID : STRING
        DECLARE Name : STRING
        DECLARE Age : INTEGER
        DECLARE Gender : CHAR
ENDTYPE

// Declaring a variable of our defined datatype
DECLARE MyStudent : Students

// Performing operations with the variable
MyStudent.Name ← "Ali"
MyStudent.StudentID ← "MR1001"
MyStudent.Age ← 17
MyStudent.Gender ← 'M'

// Declaring an array of our defined datatype
DECLARE MyData : ARRAY[1:10] OF Students

// Accessing 5th element and storing name there
MyData[5].Name ← "Zuhaib"

// Copying our data to 8th element
MyData[8].Name ← MyData[5].Name
```

Sets

```
// Format to define datatype
TYPE <set-identifier> = SET OF <Base_Datatype>

// Format to define a variable
DEFINE <identifier> (value1, value2, value3, ...) : <set-identifier>

// Example
TYPE letters = SET OF CHAR
DEFINE vowel ('a', 'e', 'i', 'o', 'u') : letters
```

Non-composite data types

Enumerated

// Format to define datatype
TYPE <identifier> = (value1, value2, value3, ...)

// Example
TYPE DaysOfWeek = (Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday)
// Notice we didn't used inverted commas with elements as these aren't strings instead our defined elements

DECLARE Today : DaysOfWeek
DECLARE Tomorrow : DaysOfWeek

Today ← Monday
Tomorrow ← Today + 1 // Now this is set to Tuesday, this operations works as numbers are used in this datatype

Pointer

A **pointer data type** is used to reference a memory location. This data type needs to have information about the type of data that will be stored in the memory location.

A **pointer data type** is a special variable that stores the memory address of another variable or function, rather than the actual data itself. By holding a memory address, a pointer allows for direct access and manipulation of data in memory, which is crucial for tasks like managing dynamic data structures (linked lists, trees), efficiently accessing array elements, and enabling call-by-reference in functions.

// Format to define datatype
TYPE <pointer> = ^<Typename>

// <Typename> is the type of data to be found in the memory location, for example, Integer, Real, User-Defined Data Type etc.

// '^' shows that the type being declared is a pointer

Insertion Sort

Pseudocode:

```
NumberOfitems = 5

FOR Pointer ← 2 TO NumberOfitems

        ItemToBeInserted ← MyList[Pointer]
        CurrentItem ← Pointer – 1

        WHILE (MyList[CurrentItem] > ItemToBeInserted) AND
                (CurrentItem > 0) DO

                MyList[CurrentItem + 1] ← MyList[CurrentItem]

                CurrentItem ← CurrentItem – 1
        ENDWHILE

        MyList[CurrentItem + 1] ← ItemToBeInserted
NEXT
```

Trace Table(Dry Run):

Pointer	ItemToBeInserted	CurrentItem	MyList[]				
			1	**2**	**3**	**4**	**5**
			55	23	15	20	35
2	23	1		55			
		0	23				
3	15	2			55		
		1		23			
		0	15				
4	20	3				55	
		2			23		
		1		20			
5	35	4					55
		3				35	

Binary Search

```
PROCEDURE BinarySearch(Num : INTEGER)
    DECLARE Top, Bottom, Middle : INTEGER
    DECLARE Found : BOOLEAN

    Found ← FALSE
    Top ← 1
    Bottom ← 10

    WHILE Found = FALSE AND Top <= Bottom DO
        Middle ← INT((Top + Bottom)/2)
        IF Num = MyList[Middle] THEN
            Found ← TRUE
        ELSEIF Num > MyList[Middle] THEN
            Top ← Middle + 1

        ELSE
            Bottom ← Middle – 1
        ENDIF
    ENDWHILE

    IF Found THEN
        OUTPUT "Value found at", Middle
    ELSE
        OUTPUT "Value not found"
    ENDIF
ENDPROCEDURE
```

```
FUNCTION BinarySearch(Num : INTEGER) RETURNS INTEGER
    DECLARE Top, Bottom, Middle : INTEGER

    Top ← 1
    Bottom ← 10

    WHILE Top <= Bottom DO
        Middle ← INT((Top + Bottom)/2)
        IF Num = MyList[Middle] THEN
            RETURN Middle
        ELSEIF Num > MyList[Middle] THEN
            Top ← Middle + 1

        ELSE
            Bottom ← Middle – 1
        ENDIF
    ENDWHILE

    RETURN -1
ENDFUNCTION

// Calling the function
DECLARE X : INTEGER

OUTPUT "Enter a number to search"
INPUT Number

X ← BinarySearch(Number)

IF X = -1 THEN
    OUTPUT "Value not found"
ELSE
    OUTPUT "Value found at: ", X
ENDIF
```

Pseudocode for Random File

```
TYPE Students
        DECLARE StudentID : INTEGER
        DECLARE Name : STRING
        DECLARE Age : INTEGER
        DCELARE Gender : CHAR
ENDTYPE

// Declared Global Variable
DECLARE MyStudent : Students

PROCEDURE AddRecord()
        OPENFILE "MyData.dat" FOR RANDOM
        INPUT MyStudent.StudentID
        INPUT MyStudent.Name
        INPUT MyStudent.Age
        INPUT MyStudent.Gender

        Index ← Hash(MyStudent.StudentID)
        SEEK "MyData.dat", Index
        PUTRECORD "MyData.dat", MyStudent
        CLOSEFILE "MyData.dat"
ENDPROCEDURE

PROCEDURE SearchRecord(SearchID : INTEGER)
        OPENFILE "MyData.dat" FOR RANDOM
        Index ← Hash(SearchID)
        SEEK "MyData.dat", Index
        GETRECORD "MyData.dat", MyStudent

        OUTPUT MyStudent.StudentID
        OUTPUT MyStudent.Name
        OUTPUT MyStudent.Age
        OUTPUT MyStudent.Gender

        CLOSEFILE "MyData.dat"
ENDPROCEDURE
```

Pseudocode for Stack

```
// Declaring Global Variables
CONSTANT Max = 5
DECLARE MyList : ARRAY[1:Max] OF STRING
DECLARE TOS : INTEGER

-----------------------------------------------------------------------------------------------------------------------
PROCEDURE InitialiseStack()
        DECLARE Index : INTEGER
        TOS ← 0
        FOR Index ← 1 TO Max
                MyList[Index] ← “”
        NEXT
ENDPROCEDURE


FUNCTION Push(NewItem : STRING) RETURNS BOOLEAN
        IF TOS = Max THEN
                RETURN FALSE
        ELSE
                TOS ← TOS + 1
                MyList[TOS] ← NewItem
                RETURN TRUE
        ENDIF
ENDFUNCTION


FUNCTION Pop() RETURNS STRING
        DECLARE Temp : STRING
        IF TOS = 0 THEN
                RETURN “EMPTY”
        ELSE
                Temp ← MyList[TOS]
                TOS ← TOS – 1
                RETURN Temp
        ENDIF
ENDFUNCTION
```

```
CALL    InitialiseStack()
CALL    Push('A')
CALL    Push('B')
CALL    Pop()
```

TOS = ~~0~~ ~~1~~ ~~2~~ 1

MyList[]

1	A
2	~~B~~ " "
3	" "
4	" "
5	" "

Zuhaib Abdul Khalique | zuhaib.khaliq@gmail.com

Pseudocode for Queue

```
// Declaring Global Variables
CONSTANT Max = 5
DECLARE MyList : ARRAY[1:Max] OF STRING
DECLARE Head, Tail : INTEGER

-----------------------------------------------------------------------------------------------------------------
PROCEDURE InitialiseQueue()
        DECLARE Index : INTEGER
        Head ← 0
        Tail ← 0
        FOR Index ← 1 TO Max
                MyList[Index] ← “”
        NEXT
ENDPROCEDURE

FUNCTION Enqueue(NewItem : STRING) RETURNS BOOLEAN
        IF (Head = 1 AND Tail = 5) OR (Head = Tail + 1) THEN
                RETURN FALSE
        ELSE
                IF Head = 0 AND Tail = 0 THEN
                        Head ← 1
                        Tail ← 1
                ELSEIF Tail = Max THEN
                        Tail ← 1
                ELSE
                        Tail ← Tail + 1
                ENDIF
                MyList[Tail] ← NewItem
                RETURN TRUE
        ENDIF
ENDFUNCTION
```

- **Dequeue function on next page**

```
FUNCTION Dequeue() RETURNS STRING
    IF Head = 0 AND Tail = 0
        RETURN "EMPTY"
    ELSE
        Temp ← MyList[Head]
        IF Head = Tail THEN
            Head ← 0
            Tail ← 0
        ELSEIF Head = Max THEN
            Head ← 1
        ELSE
            Head ← Head + 1
        ENDIF
        RETURN Temp
    ENDIF
ENDFUNCTION
```

Binary Tree Pseudocode

```
TYPE Tree
        DECLARE Data : STRING
        DECLARE LeftPtr : INTEGER
        DECLARE RightPtr : INTEGER
ENDTYPE
DECLARE MyList : ARRAY[1:10] OF Tree
DECLARE RootPtr, FreePtr : INTEGER
CONSTANT NullPtr = 0
```

```
PROCEDURE InitialiseTree()
        DECLARE Index : INTEGER
        RootPtr ← NullPtr
        FreePtr ← 1
        FOR Index ← 1 To 9
                MyList[Index].RightPtr ← Index + 1
        NEXT Index
        MyList[10].RightPtr ← NullPtr
ENDPROCEDURE

PROCEDURE SearchTree(Item : STRING)
        Found ← False
        CurrentPtr ← RootPtr

        WHILE CurrentPtr <> NullPtr AND Found = FALSE
                IF Item = MyList[CurrentPtr].Data THEN
                        Found = FALSE
                ELSEIF Item > MyList[CurrentPtr].Data THEN
                        CurrentPtr ← MyList[CurrentPtr].RightPtr
                ELSE
                        CurrentPtr ← MyList[CurrentPtr].LeftPtr
                ENDIF
        ENDWHILE

        IF Found = TRUE THEN
                OUTPUT "Data found at", CurrentPtr
        ELSE
                OUTPUT "Data not found"
        ENDIF
ENDPROCEDURE
```

```
PROCEDURE InsertTree(NewItem : STRING)
    IF FreePtr = NullPtr THEN
        OUTPUT "Tree is full"
    ELSE
        Temp ← FreePtr
        FreePtr ← MyList[FreePtr].RightPtr
        MyList[Temp].Data ← NewItem
        MyList[Temp].LeftPtr ← NullPtr
        MyList[Temp].RightPtr ← NullPtr
        IF RootPtr = NullPtr THEN
            RootPtr ← Temp
        ELSE
            CurrentPtr ← RootPtr

            REPEAT
                PreviousPtr ← CurrentPtr
                IF NewItem > MyList[CurrentPtr].Data THEN
                    TurnRight ← TRUE
                    CurrentPtr ← MyList[CurrentPtr].RightPtr
                ELSE
                    TurnRight ← FALSE
                    CurrentPtr ← MyList[CurrentPtr].LeftPtr
                ENDIF
            UNTIL CurrentPtr = NullPtr

            IF TurnRight = TRUE THEN
                MyList[PreviousPtr].RightPtr ← Temp
            ELSE
                MyList[PreviousPtr].LeftPtr ← Temp
            ENDIF
        ENDIF
    ENDIF
ENDPROCEDURE
```

Link List Pseudocode

```
TYPE List
        DECLARE Data : STRING
        DECLARE Pointer : INTEGER
ENDTYPE
DECLARE MyList : ARRAY[1:10] OF List
DECLARE StartPtr, FreePtr : INTEGER
CONSTANT NullPtr = 0
```

```
PROCEDURE InitialiseList()
        DECLARE Index : INTEGER
        StartPtr ← NullPtr
        FreePtr ← 1
        FOR Index ← 1 To 9
                MyList[Index].Pointer ← Index + 1
        NEXT Index
        MyList[10].Pointer ← NullPtr
ENDPROCEDURE

PROCEDURE SearchList(Item : STRING)
        Found ← FALSE
        CurrentPtr ← StartPtr

        WHILE Found = FALSE AND CurrentPtr <> NullPtr AND Item > MyList[CurrentPtr].Data
                IF Item = MyList[CurrentPtr].Data THEN
                        Found ← TRUE
                ELSE
                        CurrentPtr ← MyList[CurrentPtr].Pointer
                ENDIF
        ENDWHILE

        IF Found = TRUE THEN
                OUTPUT "Data found at", CurrentPtr
        ELSE
                OUTPUT "Data not found"
        ENDIF
ENDPROCEDURE
```

```
PROCEDURE InsertList(NewItem : STRING)

    IF FreePtr = NullPtr THEN
        OUTPUT "List is full"
    ELSE
        Temp ← FreePtr
        FreePtr ← MyList[FreePtr].Pointer
        MyList[Temp].Data ← NewItem

        IF StartPtr = NullPtr OR NewItem < MyList[StartPtr].Data THEN
            MyList[Temp].Pointer ← StartPtr
            StartPtr ← Temp
        ELSE
            CurrentPtr ← StartPtr

            WHILE CurrentPtr <> NullPtr AND NewItem > MyList[CurrentPtr].Data
                PreviousPtr ← CurrentPtr
                CurrentPtr ← MyList[CurrentPtr].Pointer
            ENDWHILE

            MyList[Temp].Pointer ← CurrentPtr
            MyList[PreviousPtr].Pointer ← Temp
        ENDIF
    ENDIF
ENDPROCEDURE
```

```
PROCEDURE DeleteList(Item : STRING)

    Found ← FALSE
    CurrentPtr ← StartPtr

    WHILE Found = FALSE AND CurrentPtr <> NullPtr
        IF Item = MyList[CurrentPtr].Data THEN
            Found ← TRUE
        ELSE
            PreviousPtr ← CurrentPtr
            CurrentPtr ← MyList[CurrentPtr].Pointer
        ENDIF
    ENDWHILE

    IF CurrentPtr <> NullPtr THEN
        IF StartPtr = CurrentPtr THEN
            StartPtr ← MyList[CurrentPtr].Pointer
        ELSE
            MyList[PreviousPtr].Pointer ← MyList[CurrentPtr].Pointer
        ENDIF
        MyList[CurrentPtr].Pointer ← FreePtr
        FreePtr ← CurrentPtr
    ENDIF
ENDPROCEDURE
```

Object-Oriented Programming (OOP)

Object-Oriented Programming is a paradigm based on the concept of **classes** and **objects**.

- **Class**: A collection of **Methods** (Functions & Procedures) and **Attributes/Properties** (Variables).
- **Object**: An instance defined from a given class; several objects can be defined from the same class.

Basic Structure of a Class

In OOP, properties and methods are assumed to be **PUBLIC** unless specified as **PRIVATE**. Private members are only accessible within the class itself.

Format to define a class:

```
CLASS <ClassName>
   PRIVATE <AttributeName> : <DataType>

   // Constructor
   PUBLIC PROCEDURE NEW(<Parameters>)
      <Statements>
   ENDPROCEDURE

   // Methods
   PUBLIC PROCEDURE <MethodName>(<Parameters>)
      <Statements>
   ENDPROCEDURE
ENDCLASS
```

Key OOP Concepts

1. The Constructor (NEW)

The constructor is a special procedure used to initialize an object when it is created. In pseudocode, this procedure must always be named **NEW**.

2. Inheritance

Inheritance allows a **subclass** to use properties and methods of a **superclass**.

- **INHERITS**: Keyword used to denote inheritance.
- **SUPER**: Keyword used to call the constructor or methods of the parent class.

3. Working with Objects

- **Creating an Object**: Use the NEW keyword followed by the class name and any required parameters.
- **Calling Methods**: Use **dot notation** to access public methods or properties.

Example - OOP in Practice

Defining the Classes:

```
CLASS Pet
   PRIVATE Name : STRING

   PUBLIC PROCEDURE NEW(GivenName : STRING)
      Name ← GivenName
   ENDPROCEDURE
ENDCLASS

CLASS Cat INHERITS Pet
   PRIVATE Breed : STRING

   PUBLIC PROCEDURE NEW(GivenName : STRING, GivenBreed : STRING)
      SUPER.NEW(GivenName) // Calls Parent Constructor
      Breed ← GivenBreed
   ENDPROCEDURE
ENDCLASS
```

Using the Classes:

```
// Create a new object
DECLARE MyCat : Cat
MyCat ← NEW Cat("Kitty", "Shorthaired")

// Example method call (assuming a GetName method existed)
OUTPUT MyCat.GetName()
```